

Revisão de Código

BR Gorjeta — Portal Web

Análise de Segurança, Arquitetura, Clean Code e Identidade Visual

Data: 09/03/2026

Categoria	Critical	High	Medium	Low	Total
Segurança (Backend)	4	7	7	6	24
Arquitetura (Frontend)	0	4	8	13	25
Identidade Visual	0	4	4	2	10
TOTAL	4	15	19	21	59

1. Segurança

1.1 Vulnerabilidades Críticas

[CRITICAL] JWT Secret Hardcoded

Arquivo: `api/src/services/auth.service.js`

Problema: O secret do JWT possui fallback hardcoded: `'brg-secret-key-2026'`. Se a variável de ambiente não for definida, qualquer pessoa com acesso ao código pode forjar tokens.

Correção: Remover o fallback. Exigir `JWT_SECRET` como variável de ambiente obrigatória. Validar na inicialização do servidor.

[CRITICAL] Senha com Hash MD5

Arquivo: `api/src/services/auth.service.js`

Problema: Senhas são hashadas com MD5, que é criptograficamente quebrado. Vulnerável a rainbow tables e ataques de colisão.

Correção: Substituir por `bcrypt` ou `Argon2`. Implementar salt adequado. Migrar senhas existentes progressivamente no login.

[CRITICAL] Upload de Arquivo sem Validação de Conteúdo

Arquivo: `api/src/middlewares/upload.js`

Problema: Validação de upload baseia-se apenas em MIME type e extensão, ambos facilmente falsificáveis pelo cliente.

Correção: Validar magic numbers (assinatura do arquivo). Adicionar scan de vírus. Renomear arquivos com UUID. Limitar uploads por usuário/dia.

[CRITICAL] Sem Rate Limiting no Login

Arquivo: `api/src/app.js`

Problema: Nenhum rate limiting configurado. API vulnerável a ataques de força bruta e credential stuffing no endpoint de login.

Correção: Adicionar `express-rate-limit`: 5 tentativas/15min no login, 100 req/min nas demais rotas.

1.2 Vulnerabilidades Altas

[HIGH] Sem Proteção CSRF

Arquivo: `api/src/app.js`

Problema: Nenhuma validação de token CSRF em requisições que alteram estado (POST, PUT, PATCH, DELETE).

Correção: Implementar `csrf` middleware ou validar Origin/Referer headers.

[HIGH] CORS Permissivo

Arquivo: `api/src/app.js`

Problema: Origin do CORS vem direto de variável de ambiente sem validação. Se configurado como "*", qualquer site acessa a API.

Correção: Validar origins contra whitelist. Não aceitar wildcard em produção.

[HIGH] Bypass de Autorização entre Clientes

Arquivo: `api/src/middlewares/auth.js`

Problema: O middleware autentica o token mas não verifica se o `cliente_id` da requisição pertence ao usuário logado. Um token válido do cliente A pode acessar dados do cliente B.

Correção: Comparar `cliente_id` do JWT com o da requisição em todas as rotas protegidas.

[HIGH] Parâmetros de Rota sem Validação

Arquivo: `api/src/controllers/colaboradores.controller.js`

Problema: Parâmetros `:id` são parseados com `parseInt()` sem validação de bounds ou tipo.

Correção: Validar que IDs são inteiros positivos antes de processar. Rejeitar valores inválidos com 400.

1.3 Vulnerabilidades Médias

[MEDIUM] Dados Sensíveis no JWT

Arquivo: `api/src/services/auth.service.js`

Problema: CNPJ completo, email e status administrativo retornados no token JWT.

Correção: Minimizar claims no JWT. Incluir apenas `usr_codigo` e `crd_cli_id`. Buscar dados adicionais via endpoints protegidos.

[MEDIUM] Sem Headers CSP (Content Security Policy)

Arquivo: `api/src/app.js`

Problema: Helmet está configurado mas sem CSP explícito, permitindo XSS via scripts inline.

Correção: Configurar directives do CSP: `defaultSrc`, `scriptSrc`, `styleSrc`, `imgSrc`, `frameSrc`.

[MEDIUM] Erro Pode Expor Detalhes Internos

Arquivo: `api/src/middlewares/errorHandler.js`

Problema: O objeto `details` do erro é retornado ao cliente sem sanitização.

Correção: Criar whitelist de campos permitidos no `details`. Sanitizar antes de retornar.

2. Arquitetura e Clean Code

2.1 Backend — Pontos Positivos

O backend segue um padrão consistente de Controller → Service → Repository, o que é uma boa prática de separação de responsabilidades. As queries SQL são parametrizadas na maioria dos casos, e o middleware de erro centralizado é bem implementado.

2.2 Frontend — Problemas Altos

[HIGH] Memory Leak no AuthContext

Arquivo: `frontend/src/contexts/AuthContext.jsx`

Problema: `useEffect` que verifica token salvo não tem `cleanup`. Se o componente desmontar durante a chamada API, atualização de estado causa memory leak.

Correção: Adicionar `AbortController` no `useEffect` com `cleanup` no `return`.

[HIGH] Memory Leak nos Gráficos Chart.js

Arquivo: `frontend/src/pages/DashboardPage.jsx`

Problema: Instâncias `Chart.js` são criadas no `useEffect` mas o `cleanup` só roda quando dados mudam, não quando o componente desmonta.

Correção: Adicionar `useEffect` separado com `cleanup []` para destruir charts no `unmount`.

[HIGH] Redirect 401 com Hard Reload

Arquivo: `frontend/src/services/api.js`

Problema: Ao receber 401, executa `window.location.href = "/login"`, causando reload completo da página e perda de estado React.

Correção: Usar `React Router navigate()` ou emitir evento customizado que o `AuthContext` escuta.

[HIGH] Nenhum PropTypes/TypeScript

Arquivo: Todos os componentes

Problema: Nenhum componente define `PropTypes` ou usa `TypeScript`. Sem validação de props em tempo de compilação.

Correção: Adicionar `PropTypes` a todos os componentes ou migrar para `TypeScript`.

2.3 Frontend — Problemas Médios

[MEDIUM] Inline Styles Excessivos

Arquivo: `ListaColaboradoresGestao`, `FormColaborador`, `Layout`

Problema: Estilos inline espalhados por todos os componentes. Dificulta refatoração, impede media queries e viola separação de concerns.

Correção: Extrair para CSS modules ou arquivo CSS centralizado.

[MEDIUM] Magic Numbers e Strings Hardcoded

Arquivo: [ListaColaboradoresGestao.jsx](#), [FormColaborador.jsx](#)

Problema: Valores como `limit=15`, `situacao_id=1` (ATIVO), `situacao_id=2` (BLOQUEADO) estão hardcoded.

Correção: Criar arquivo de constantes (`constants.js`) com enums e configurações.

[MEDIUM] useState Excessivo no FormColaborador

Arquivo: [frontend/src/components/FormColaborador.jsx](#)

Problema: 11 hooks `useState` separados para gerenciar o formulário. Difícil de manter.

Correção: Usar `useReducer` para estado complexo do formulário.

[MEDIUM] Prop Drilling em 3 Níveis

Arquivo: [ColaboradoresPage](#) → [ColaboradoresManager](#) → [ListaColaboradoresGestao](#)

Problema: `clienteId` e `login` são passados por 3 camadas de componentes.

Correção: Usar `React Context` dedicado (`ClienteContext`) para evitar prop drilling.

[MEDIUM] Código Duplicado (CPF, Datas)

Arquivo: [FormColaborador](#), [ListaColaboradoresGestao](#), [Layout](#)

Problema: Funções de formatação de CPF e datas são repetidas em 3+ componentes.

Correção: Extrair para `utils/formatters.js` e reutilizar.

3. Identidade Visual

3.1 Cores — Inconsistências

Uso	Cor Atual	Arquivo	Problema
Sidebar/Header	#421847	Layout.css	Diferente do roxo principal
Login/Dashboard	#4A1D4F	LoginPage.css, Dashboard.css	Roxo principal
Background página	#f0f2f5	Layout.css	—
Background CreditoForm	#f4f6f9	CreditoForm.css	Diferente do Layout
Background body	#f5f7fb	index.css	Terceira variação de cinza
Rosa hover Login	#e8557a	LoginPage.css	Diferente do CreditoForm
Rosa hover Credito	#e85a7e	CreditoForm.css	Diferente do Login

Recomendação: Unificar para #4A1D4F como roxo principal em todos os arquivos. Definir uma única cor de fundo: #f0f2f5.

3.2 Border-Radius — 6 Valores Diferentes

Valor	Onde Usado
6px	Inputs LoginPage, sidebar, CreditoForm
8px	Dropdown header, CreditoForm --radius-sm
10px	Cards Dashboard, CreditoForm --radius
14px	Welcome banner, Filter bar
15px	Login card
20px	Date inputs Dashboard, botão Filtrar

Recomendação: Padronizar em 3 valores: --radius-sm: 6px, --radius-md: 10px, --radius-lg: 14px.

3.3 Tipografia

Font-family está consistente (Nunito), porém CreditoForm.css define Inter como fonte, conflitando. Tamanhos de fonte variam entre px e rem sem escala definida. Pesos de botões variam (600, 700, bold).

Recomendação: Criar escala tipográfica com variáveis CSS (--text-xs a --text-xl) e usar apenas Nunito.

3.4 Responsividade

Breakpoints inconsistentes: 575px, 700px, 768px, 850px, 1100px. Layout.css não tem nenhuma regra responsiva — sidebar não colapsa em mobile. Não há menu hamburger.

Recomendação: Padronizar breakpoints (480px, 768px, 1024px). Implementar sidebar colapsável com hamburger menu em mobile.

3.5 Acessibilidade

Items inativos do sidebar têm contraste ~3.5:1 (abaixo do mínimo WCAG AA de 4.5:1). Faltam focus states visíveis em elementos interativos. Dropdown do header não fecha com ESC nem com clique fora. Nenhum aria-label nos ícones.

4. Plano de Ação Prioritário

4.1 Prioridade Imediata (Segurança)

#	Ação	Risco	Esforço
1	Substituir MD5 por bcrypt nas senhas	CRITICAL	Médio
2	Remover JWT secret hardcoded	CRITICAL	Baixo
3	Adicionar rate limiting (login + API)	CRITICAL	Baixo
4	Validar conteúdo real dos uploads	CRITICAL	Médio
5	Verificar cliente_id do JWT vs requisição	HIGH	Baixo
6	Configurar CORS com whitelist	HIGH	Baixo

4.2 Prioridade Alta (Estabilidade)

#	Ação	Impacto
7	Corrigir memory leaks (AuthContext + Chart.js)	Estabilidade
8	Substituir redirect 401 por React Router	UX
9	Adicionar PropTypes em todos componentes	Manutenção
10	Criar arquivo de constantes (IDs, limites)	Clean Code
11	Extrair funções duplicadas para utils/	Clean Code

4.3 Prioridade Média (Visual e UX)

#	Ação	Impacto
12	Unificar cores (#4A1D4F único, fundo único)	Visual
13	Criar design tokens centralizados (CSS vars)	Visual
14	Padronizar border-radius (3 valores)	Visual
15	Implementar responsividade no sidebar	UX Mobile
16	Melhorar contraste para WCAG AA	Acessibilidade
17	Extrair inline styles para CSS	Manutenção
18	Refatorar FormColaborador com useReducer	Clean Code